

React Native & Expo

Développer des applications mobiles avec JavaScript

React Native	Expo	Composants	Hooks
--------------	------	------------	-------

StyleSheet	FlatList	Animations
------------	----------	------------

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Comprendre l'architecture de React Native et son modèle de rendu
- ✓ Maîtriser les composants natifs fondamentaux et leur comportement mobile
- ✓ Gérer l'état local et les effets avec useState et useEffect
- ✓ Construire des listes performantes avec FlatList et SectionList
- ✓ Gérer les styles avec StyleSheet et les layouts avec Flexbox
- ✓ Accéder aux APIs natives du téléphone via les modules Expo

Table des matières

Chapitre 1 — Architecture de React Native

- 1.1 Du web au mobile
- 1.2 Le pont JavaScript-natif
- 1.3 Hermes et le nouveau moteur
- 1.4 Expo — simplifier le développement

Chapitre 2 — Les composants fondamentaux

- 2.1 View, Text, Image, ScrollView
- 2.2 TouchableOpacity et Pressable
- 2.3 TextInput et formulaires
- 2.4 Modal et SafeAreaView

Chapitre 3 — État, effets et contexte

- 3.1 useState et useReducer
- 3.2 useEffect — cycle de vie
- 3.3 useContext — état global
- 3.4 Hooks personnalisés

Chapitre 4 — Listes et performance

- 4.1 FlatList — virtualisation
- 4.2 SectionList
- 4.3 Optimisations : memo, useCallback, useMemo
- 4.4 Images et cache

Chapitre 5 — Styles, layout et APIs natives

- 5.1 StyleSheet et dimensions
- 5.2 Flexbox sur mobile
- 5.3 Thèmes et dark mode

5.4 Modules Expo essentiels

Architecture de React Native

1.1 Du web au mobile

React Native est un framework créé par Meta (2015) qui permet d'écrire des applications mobiles iOS et Android en JavaScript/TypeScript avec la syntaxe React. Contrairement à une WebView (qui charge une page web dans un navigateur embarqué), React Native compile les composants React en véritables composants natifs. Un React Native devient une UIView iOS ou un android.view.View Android. Les performances sont proches du natif, et l'interface respecte les conventions visuelles de chaque plateforme.

React pour le web et React Native partagent la même philosophie (composants, props, hooks, état) mais les composants de base sont différents. Sur le web, vous utilisez div, span, button. En React Native, les équivalents sont View, Text, Pressable. La bonne nouvelle : si vous connaissez React, vous connaissez déjà 80% de React Native.

1.2 Le pont JavaScript-natif

L'architecture classique de React Native utilise un 'pont' (bridge) entre le thread JavaScript et le thread natif de l'interface. Les instructions de rendu sont sérialisées en JSON et transmises de façon asynchrone. Cette architecture a une limitation : le bridge est un goulot d'étranglement pour les interactions haute fréquence (animations à 60 fps, gestes complexes). La nouvelle architecture (JSI — JavaScript Interface) remplace le bridge par une interface C++ directe, éliminant la sérialisation JSON.

Définition — Expo

Expo est un ensemble d'outils et de services construit sur React Native. Il simplifie radicalement la mise en place d'un projet, l'accès aux APIs natives (caméra, notifications, position GPS), et le déploiement. Expo Go permet de tester rapidement sans compiler. EAS Build compile des APK/IPA natifs complets. La majorité des projets professionnels utilisent Expo aujourd'hui.

1.3 Expo — simplifier le développement

Expo résout plusieurs problèmes pénibles du développement React Native natif : la configuration d'Android Studio et Xcode, la gestion des dépendances natives, la signature des apps pour les stores. Expo propose deux modes : Expo Go (app de développement rapide, limité aux modules Expo) et Development Build (app native complète avec expo-dev-client — recommandé pour les projets sérieux).

```
# Créer un nouveau projet Expo avec TypeScript
```

```
npx create-expo-app@latest mon-app --template blank-typescript
```

Démarrer le serveur de développement

`npx expo start`

Démarrer en mode dev client (après un EAS build)

`npx expo start --dev-client`

Structure d'un projet Expo

`mon-app/`

`app/` # Routes (Expo Router)

`components/` # Composants réutilisables

`lib/` # Utilitaires, clients API

`assets/` # Images, fonts

`app.json` # Configuration Expo

`eas.json` # Configuration EAS Build

Les composants fondamentaux

2.1 View, Text, Image, ScrollView

Tout ce qui est visible dans une application React Native est composé de ces primitives. View est le conteneur universel — l'équivalent d'un div en HTML. Text est le seul composant pour afficher du texte (contrairement au web, vous ne pouvez pas mettre du texte directement dans une View). Image affiche des images locales ou distantes. ScrollView est un conteneur scrollable qui rend tous ses enfants simultanément — à ne pas utiliser pour de grandes listes.

```
import { View, Text, Image, ScrollView, StyleSheet } from 'react-native';

export function ProductCard({ product }) {
  return (
    <View style={styles.card}>
      <Image
        source={{ uri: product.imageUrl }}
        style={styles.image}
        resizeMode='cover'
      />
      <View style={styles.info}>
        <Text style={styles.name}>{product.name}</Text>
        <Text style={styles.price}>
          {(product.price / 100).toFixed(0)} FCFA
        </Text>
      </View>
    </View>
  );
}
```

2.2 TouchableOpacity et Pressable

Tout élément interactif doit être enveloppé dans un composant tactile. TouchableOpacity réduit l'opacité de ses enfants quand on appuie dessus — un feedback visuel universel et apprécié sur mobile. Pressable est plus récent et plus flexible : il expose l'état de pression et supporte des styles différents pour chaque état (pressed, focused, disabled). À préférer pour les nouveaux projets.

2.3 TextInput et formulaires

TextInput est le composant pour la saisie de texte. Il expose de nombreuses props pour configurer le clavier (keyboardType), la capitalisation, la correction automatique, et le comportement au focus. Sur mobile, le clavier virtuel peut masquer le champ actif — le wrapper KeyboardAvoidingView ou les bibliothèques comme react-native-keyboard-controller règlent ce problème.

```
import { TextInput, KeyboardAvoidingView, Platform } from 'react-native';

// Formulaire avec gestion du clavier
export function LoginForm({ onSubmit }) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  return (
    <KeyboardAvoidingView
      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
    >
      <TextInput
        value={email}
        onChangeText={setEmail}
        placeholder='Email'
        keyboardType='email-address'
        autoCapitalize='none'
        autoCorrect={false}
      />
      <TextInput
        value={password}
        onChangeText={setPassword}
        placeholder='Mot de passe'
        secureTextEntry // Masquer le texte
      />
    </KeyboardAvoidingView>
  );
}
```


État, effets et contexte

3.1 useState et useReducer

useState est le hook fondamental pour l'état local d'un composant. Il retourne une paire [valeur, setter]. Quand le setter est appelé, React re-rend le composant avec la nouvelle valeur. useReducer est une alternative à useState pour les états complexes avec plusieurs sous-valeurs ou des transitions d'état interdépendantes — il suit le pattern Redux (action → reducer → nouveau state).

```
// useState – cas simple
const [messages, setMessages] = useState<Message[]>([]);
const [loading, setLoading] = useState(false);
const [error, setError] = useState<string | null>(null);

// useReducer – état complexe (ex: inbox avec filtres et pagination)
type InboxState = {
  conversations: Conversation[];
  loading: boolean;
  filter: 'all' | 'unread' | 'archived';
  page: number;
};

type InboxAction =
  | { type: 'LOAD_START' }
  | { type: 'LOAD_SUCCESS'; data: Conversation[] }
  | { type: 'SET_FILTER'; filter: InboxState['filter'] }
  | { type: 'NEXT_PAGE' };

function inboxReducer(state: InboxState, action: InboxAction): InboxState {
  switch (action.type) {
    case 'LOAD_START': return { ...state, loading: true };
    case 'LOAD_SUCCESS': return { ...state, loading: false, conversations: action.data };
    case 'SET_FILTER': return { ...state, filter: action.filter, page: 1 };
    default: return state;
  }
}
```

3.2 useEffect — cycle de vie

`useEffect` est le hook pour les effets de bord : requêtes réseau, souscriptions, timers, accès aux APIs natives. Il s'exécute après le rendu. Le tableau de dépendances contrôle quand il se réexécute. La fonction de nettoyage (cleanup) retournée par `useEffect` s'exécute avant le prochain effet et avant le démontage du composant — indispensable pour les souscriptions.

useEffect avec gestion des race conditions

```
useEffect(() => {
  let cancelled = false;

  async function fetchConversations() {
    setLoading(true);
    const { data, error } = await supabase
      .from('conversations')
      .select('*')
      .order('updated_at', { ascending: false });

    if (!cancelled) { // Éviter les race conditions
      if (error) setError(error.message);
      else setConversations(data ?? []);
      setLoading(false);
    }
  }

  fetchConversations();

  return () => { cancelled = true; }; // Cleanup
}, []); // [] = exécuter seulement au montage
```

Listes et performance

4.1 FlatList — virtualisation

FlatList est le composant essentiel pour les listes en React Native. Contrairement à ScrollView qui rend tous ses enfants simultanément, FlatList virtualise la liste : seuls les éléments visibles à l'écran sont réellement rendus. Les éléments hors-écran sont détachés du DOM natif, économisant la mémoire et le CPU. Sur une liste de 1000 éléments, FlatList ne rend que les 15-20 visibles.

```
import { FlatList } from 'react-native';

export function ConversationList({ conversations, onPress }) {
  return (
    <FlatList
      data={conversations}
      keyExtractor={({item}) => item.id} // Clé unique obligatoire
      renderItem={({ item }) => (
        <ConversationCard
          conversation={item}
          onPress={onPress}
        />
      )}
      // Performance
      maxToRenderPerBatch={10}
      windowSize={5}
      removeClippedSubviews={true}
      // Pull-to-refresh
      refreshing={loading}
      onRefresh={handleRefresh}
      // Pagination infinie
      onEndReached={handleLoadMore}
      onEndReachedThreshold={0.2}
      // Messages inversés (chat)
      inverted
    />
  )
}
```

```
);  
}
```

4.2 Optimisations : memo, useCallback, useMemo

React re-rend un composant à chaque fois que ses props ou son état changent. Dans une FlatList avec des centaines d'items, chaque changement dans le composant parent peut déclencher le re-rendu de tous les items. React.memo enveloppe un composant et empêche son re-rendu si ses props n'ont pas changé. useCallback mémorise une fonction entre les rendus. useMemo mémorise le résultat d'un calcul coûteux.

```
import { memo, useCallback, useMemo } from 'react';  
  
// memo – éviter les re-rendus inutiles des items de liste  
const ConversationCard = memo(function ConversationCard({ conversation, onPress }) {  
// Ne se re-rend que si conversation ou onPress change  
  return <TouchableOpacity onPress={() =>  
    onPress(conversation.id)}>...</TouchableOpacity>;  
});  
  
// Dans le composant parent  
function InboxScreen() {  
  const [conversations, setConversations] = useState([]);  
  
// useCallback – la référence de onPress reste stable entre les rendus  
  const handlePress = useCallback((id: string) => {  
    router.push(`/conversation/${id}`);  
  }, []); // Pas de dépendances – ne change jamais  
  
// useMemo – calculer les non-lus une seule fois  
  const unreadCount = useMemo(  
    () => conversations.filter(c => c.unread).length,  
    [conversations] // Recalculer seulement si conversations change  
  );  
}
```

Styles, layout et APIs natives

5.1 StyleSheet et Flexbox

React Native utilise `StyleSheet.create()` pour définir les styles. Contrairement au CSS web, il n'y a pas de cascade ni d'héritage. Chaque composant gère ses propres styles. Les propriétés utilisent le camelCase (`backgroundColor`, `fontSize`) et les dimensions sont en points indépendants de la densité d'écran (density-independent pixels), ce qui garantit une apparence cohérente sur toutes les densités d'écran.

Le layout par défaut en React Native est Flexbox, avec une différence clé par rapport au web : la direction par défaut est `column` (vertical) au lieu de `row` (horizontal). Cela reflète le fait que les écrans mobiles sont orientés verticalement par défaut.

```
const styles = StyleSheet.create({
  container: {
    flex: 1, // Prend tout l'espace disponible
    backgroundColor: '#F8FAFC',
  },
  header: {
    flexDirection: 'row', // Horizontal
    alignItems: 'center', // Centre verticalement
    justifyContent: 'space-between',
    paddingHorizontal: 16,
    paddingVertical: 12,
  },
  card: {
    backgroundColor: 'FFFFFF',
    borderRadius: 12,
    padding: 16,
    marginHorizontal: 16,
    marginBottom: 8,
    // Ombres – différent sur iOS et Android
    shadowColor: 'black',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.08,
```

```
shadowRadius: 4,  
elevation: 3, // Android uniquement  
},  
});
```

5.2 Modules Expo essentiels

Expo propose une bibliothèque de modules qui wrappent les APIs natives iOS et Android dans une interface JavaScript unifiée. Ces modules sont utilisés partout dans Social Seller.

Modules Expo utilisés dans Social Seller

Module	Usage	Exemple d'utilisation
expo-router	Navigation basée sur les fichiers	Routing de l'application
expo-secure-store	Stockage chiffré (trousseau OS)	Tokens OAuth, credentials
expo-notifications	Push notifications	Alertes de nouveaux messages
expo-web-browser	Navigateur in-app	Flux OAuth (Instagram, Facebook)
expo-image	Images optimisées	Avatars, photos produit
expo-font	Chargement de polices custom	Typographie de l'app
expo-haptics	Retours haptiques	Vibration sur action importante

■ À retenir

- ✓ React Native compile en composants natifs vrais — pas de WebView
- ✓ View, Text, Image, Pressable sont les briques de base — pas de div ni de span
- ✓ FlatList virtualise les listes — ne jamais mettre une FlatList dans une ScrollView
- ✓ useEffect avec cleanup pour toutes les souscriptions et les timers
- ✓ React.memo + useCallback = pattern standard pour les items de liste performants
- ✓ StyleSheet.create() avec Flexbox column par défaut

→ ■ Exercices

Exercice 1 — Composant de carte produit

Créer un composant ProductCard réutilisable et performant.

- Props : product, onPress, onAddToCart
- Afficher image, nom, prix formaté, badge stock (en stock / rupture)
- Wrapper dans React.memo avec les bons useCallback dans le parent
- Tester sur une FlatList de 50 produits simulés

Exercice 2 — Écran de liste avec filtres

Construire un écran de liste de commandes avec filtres et pagination.

- FlatList avec pull-to-refresh et pagination infinie (onEndReached)
- Barre de filtre par statut (tous, en cours, livrés)
- useMemo pour calculer les stats (total, count par statut)
- Loading skeleton pendant le chargement initial

Exercice 3 — Formulaire complet

Créer un formulaire de création de produit avec validation.

- Champs : nom, prix, stock, description
- Validation en temps réel (champ rouge si invalide)
- KeyboardAvoidingView pour que le clavier ne masque pas les champs
- Soumettre vers l'API et afficher le résultat (succès / erreur)

Bibliographie & Ressources

Documentation

- reactnative.dev — documentation officielle React Native
- docs.expo.dev — documentation Expo
- reactnative.dev/docs/flexbox — Flexbox sur mobile

Ressources

- William Candillon — 'Can it be done in React Native?' (YouTube)
- Theo Browne — tutoriels React Native + Expo (YouTube)
- shopify.engineering/react-native-performance — optimisation avancée